

14 — Adversarial Review & QA

Objetivo: poner un par de ojos externos sobre el deliverable *antes* de entregarlo. Un answer confiado de un modelo no es un answer correcto: tres lentes ortogonales lo revisan sin verse entre sí, una cross-examination determinística separa lo confirmado de la corazonada, y la lógica de dominio se demuestra con tests reales cuya cobertura se **mide** — no se promete. "External eyes before it ships."

Esta sección documenta dos capas que corren al final del pipeline y comparten una sola filosofía: **no confiar en la afirmación, leer la evidencia**. La primera es el *adversarial review* (`web/lib/adversarial-review.ts`, patrón addyosmani/**adverse**): tres personas con una sola llamada de modelo cada una, cross-examinadas. La segunda es el *QA engine* del runtime local (`web/scripts/build-local.mjs`): vitest + coverage gobernados por una **Queen evidence review** que rebota el build si la cobertura real no llega a la barra. Las dos heredan su disciplina de los playbooks `ADVERSARIAL` y `TEST` de `web/lib/playbooks.ts`, destilados de addyosmani/agent-skills.

Todo lo que sigue está verificado contra el código real en `/Users/alvaroz/projects/2026/puglit/web`.

Archivos de referencia:

Capa	Archivo	Símbolo
Adversarial review	<code>web/lib/adversarial-review.ts</code>	<code>adversarialReview</code> , <code>reviewLens</code> , <code>synthesize</code> , <code>repairFindings</code>
Playbooks	<code>web/lib/playbooks.ts</code>	<code>ADVERSARIAL</code> , <code>TEST</code> , <code>REVIEW</code>
Test generation (finalize)	<code>web/lib/app-builder.ts</code>	<code>genTests</code>
QA engine + Queen	<code>web/scripts/build-local.mjs</code>	<code>runTests</code> , <code>runVitest</code> , <code>writeTestsFor</code> , <code>uncovered</code> , <code>COV_BAR</code>

1. Why Adversarial Review

1.1 Un compilado verde y un answer confiado mienten igual

El runtime gate (sección 07) ya garantiza que la app *arranca* y sirve sus páginas sin 5xx. Pero arrancar no es ser correcto. Una app de reservas puede bootear perfecto y aun así sumar mal el total, llamar `reservations` en el schema y `bookings` en una ruta, construir SQL por interpolación de strings, o duplicar la regla de pricing en dos handlers que con el tiempo divergen. Ninguno de esos defectos tira un 500; un `tsc --noEmit` verde no ve ninguno.

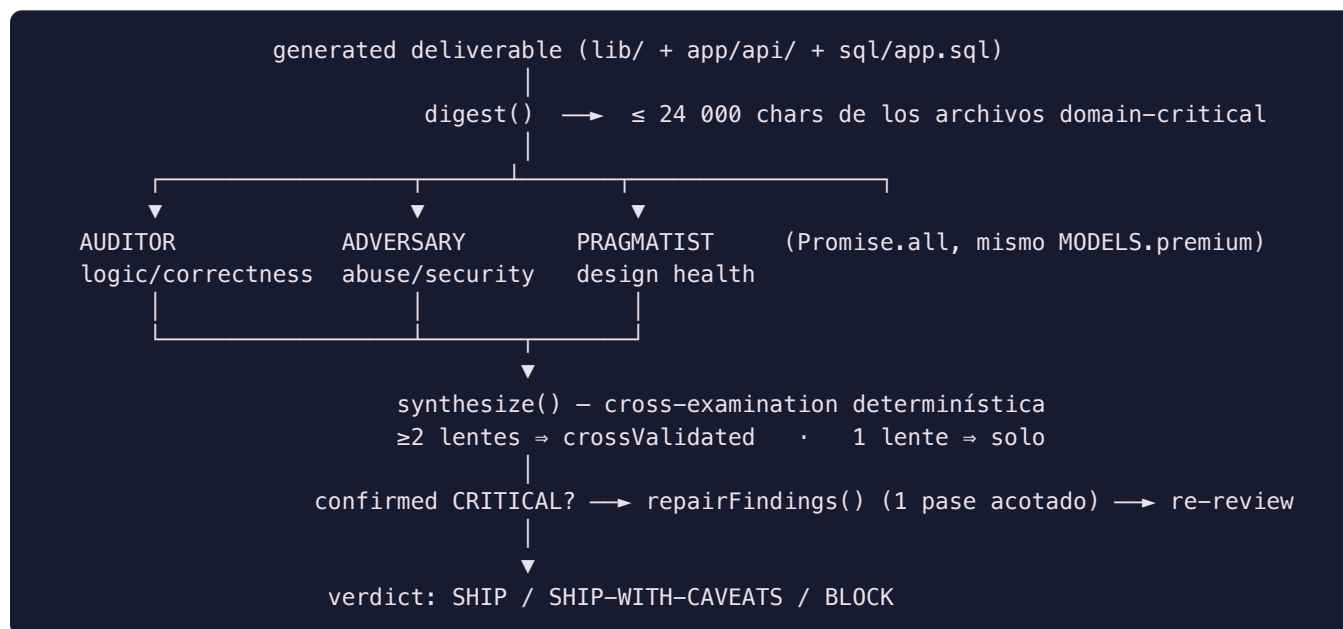
El problema de fondo es el mismo que ataca el playbook `ADVERSARIAL`:

"a confident answer is not a correct one."

Un modelo local que acaba de escribir un módulo está, por construcción, sesgado a creer que está bien. Pedirle al **mismo** modelo "¿esto está correcto?" invita al *self-consistency bias*: te confirma su propia salida. La solución del patrón addyosmani/**adverse** no es un modelo más caro, sino **adversarialidad estructural**: forzar al modelo a adoptar personas hostiles y ortogonales, y después quedarse sólo con lo que más de una persona ve.

1.2 Un modelo, tres personas — barato y sin anchoring

El header de `adversarial-review.ts` lo dice explícito: "One model, three personas: cheaper than multi-model, and the cross-lens consensus cuts the single-model anchoring bias." En vez de tres modelos distintos (caro, y en local imposible de cargar a la vez en una A40), se hace **tres llamadas al mismo MODELS.premium** con tres system prompts antagónicos y temperatura baja (0.2). El consenso entre lentes — no la potencia del modelo — es lo que corta el sesgo.



2. The Three Lenses

El núcleo del patrón son tres lentes definidas en el objeto `LENS`. Cada una es una persona con un mandato y, sobre todo, una orden de **stay-in-your-lane**: no reportar fuera de su dominio, porque la cross-examination depende de que las tres miren cosas *distintas*. Si las tres reportaran de todo, el "confirmado por ≥2" perdería su señal.

Lente	Dominio	Qué busca	Postura	Stay-in-your-lane
Auditor	LOGIC & CORRECTNESS	math de dominio mal (pricing/totales/fechas/%), edge cases sin manejar, off-by-one, mismatches de naming/schema (un concepto con dos nombres de tabla/columna), magic values hardcoded, reglas de negocio duplicadas	neutral	NO reporta security ni design-taste
Adversary	ABUSE & SECURITY	SQL injection / queries por string, falta de auth o scoping per-user (IDOR), race conditions, double-booking/double-spend, bugs de integer/precisión/overflow, input sin validar llegando a la DB, secretos en código	biased to REJECT	asume usuario hostil; NO reporta style ni logic-only
Pragmatist	DESIGN HEALTH & MAINTAINABILITY	lógica de negocio en route handlers en vez de lib/ (sin single source of truth), violaciones de DRY (regla copy-pasteada), rutas muertas, tests faltantes / baja cobertura del dominio, over-engineering	"ship sólo si correcto Y mantenible"	NO re-reporta bugs de lógica ni agujeros de security

2.1 Contrato de salida de cada lente

`reviewLens(persona, product, source)` arma el system prompt con `LENS[persona]` y obliga a un JSON estricto vía `SCHEMA` y `chatJSON`:

```
{ "verdict": "approve|conditional|reject",
  "summary": "<una oración>",
  "findings": [ { "severity": "critical|warning|info",
                  "file": "<path|null>",
                  "title": "<short noun phrase>",
                  "detail": "<2-4 oraciones con la evidencia concreta>",
                  "fix": "<remediación concreta|null>" } ] }
```

El prompt es explícito en que **un array de findings vacío + approve es la respuesta correcta si no encuentra nada** — no se premia inventar. Cada finding se sanitiza al parsearlo: severidad y verdict a un enum válido (default `warning` / `conditional`), título a 140 chars, detalle a 600, fix a 400, y se descartan findings sin título. Se topea a 12 findings por lente. La llamada nunca tira: `.catch(() => null)` y un lente caído simplemente no cuenta.

El input que ven las lentes lo produce `digest()`: filtra a los archivos domain-critical — `lib/*.ts` (no tests), `app/api/**/route.ts`, y `sql/app.sql` —, los ordena (schema primero, después `lib/`, después rutas), recorta cada archivo a 2 200 chars y corta el total en 24 000. Es deliberado: los modelos locales tienen ventanas chicas, así que la review se enfoca donde viven las reglas, no en el TSX de las páginas.

3. Cross-Examination

Las tres reviews entran a `synthesize()`, que hace la cross-examination **sin un modelo** — es lógica determinística pura. Esto importa: la decisión de qué es real no se delega a otra llamada falible, se calcula.

El mecanismo:

1. **Clave canónica por finding** (`keyOf`): el basename del archivo + una normalización del título (`norm`: lowercase, sólo alfanumérico, palabras de >3 letras, ordenadas alfabéticamente, primeras 5). Esto hace que "Pricing total is wrong" y "wrong total in pricing" colisionen en la misma clave aunque dos lentes lo hayan redactado distinto.
2. **Agrupar** todos los findings de las tres lentes por esa clave.
3. Para cada grupo, contar **cuántas lentes distintas** lo reportaron (`new Set(g.map(f => f.lens))`): - ≥ 2 lentes $\Rightarrow$ `crossValidated` — señal real, confirmada. Al detalle se le anexa `[confirmado por <auditor+adversary+...>]`. - 1 lente $\Rightarrow$ `solo` — una corazonada, un *maybe*.
4. De cada grupo se conserva el finding de **mayor severidad** (`sevRank`: critical 3 > warning 2 > info 1).
5. Ambas listas se devuelven ordenadas por severidad descendente.

```
Auditor: "wrong total" (lib/pricing.ts, critical)  γ
Adversary: "integer overflow in total" (lib/pricing.ts)  |— keyOf colisiona → crossValidated
Pragmatist: -                                           ] [confirmado por auditor+adversary

Auditor: "magic limit 50" (lib/feed.ts, warning) — 1 lente —————→ solo (hunch)
```

La filosofía es la del playbook `ADVERSARIAL`: "*a finding backed by ≥ 2 lenses is REAL; one lens's hunch is a maybe.*" La cross-examination es lo que convierte tres opiniones de un mismo modelo en una señal con menos varianza que cualquiera de las tres por separado.

4. Verdict & Bounded Repair

4.1 El pase de reparación acotado

Antes de emitir el verdict, `adversarialReview` intenta **arreglar lo que está confirmado y es crítico** — nada más. De `crossValidated` filtra los de severidad `critical` (`confirmedCritical`) y, si hay alguno, llama a `repairFindings(files, confirmedCritical)`.

`repairFindings` es minimalista por diseño:

- Sólo toca findings `critical` **con archivo nombrado**, y a lo sumo 3 (`.slice(0, 3)`).
- Sólo repara archivos `.ts` / `.tsx`.
- Por archivo, una llamada a `MODELS.code` (temp `0.1`) con el rol "*Backend Engineer fixing ONE confirmed issue in ONE file*" y el `PLAYBOOK.dev` inyectado, devolviendo el archivo entero corregido como `{ "code": ... }`. El prompt exige la **MÍNIMA** corrección: no reescribir lo que funciona, no agregar deps, no cambiar comportamiento no relacionado.
- Sólo se acepta si vuelve algo sustancial (`out.code.length > 60`); el contenido se topea a 30 000.

Si el repair tocó al menos un archivo, se hace **una re-review completa** (las tres lentes otra vez sobre el `digest` actualizado) y se re-sintetiza desde esa review fresca. No se confía en que el fix funcionó: se vuelve a medir.

4.2 La regla del verdict

```

verdict =
  stillCritical || reject >= 2          ? "BLOCK"
: (crossValidated.length
  || solo.some(f => f.severity === "critical")
  || reviews.some(r => r.verdict !== "approve")) ? "SHIP-WITH-CAVEATS"
: "SHIP"

```

Verdict	Condición
BLOCK	queda algún <code>critical</code> confirmado tras el repair (<code>stillCritical</code>), o ≥ 2 lentes votaron <code>reject</code>
SHIP-WITH-CAVEATS	hay findings cross-validados, o algún solo es <code>critical</code> , o alguna lente no aprobó
SHIP	las tres aprobaron y no quedó nada material

El resultado completo (`{ verdict, crossValidated, solo, reviews, repaired }`) se devuelve al pipeline, que lo loguea (`[adverse] verdict ...`) y registra un metric `adverse_verdict` (SHIP=1, CAVEATS=0.5, BLOCK=0) con el conteo de confirmados/solo/reparados.

4.3 Training-mode skip

El loop de reparación + re-review es la parte cara (hasta 3 fixes con `MODELS.code` + tres llamadas premium más). En el batch de brain-training (100 builds para precargar el cerebro), eso no se justifica: el guard `!process.env.PUGLIT_TRAINING_MODE` **revisa una sola vez y saltea el pase de repair + re-review**. La review sigue corriendo y dejando su señal; lo que se omite es el gasto de auto-arreglar en un contexto donde lo que importa es la diversidad de datos, no entregar ese app puntual perfecto.

4.4 Guards de entrada

`adversarialReview` también degrada con gracia: si el `digest` queda con menos de 200 chars (no hay lógica domain-critical que revisar) devuelve `null`; y si **menos de 2 lentes** respondieron (modelos caídos), también `null` — la cross-examination no tiene sentido con una sola opinión.

5. QA Engine

Mientras el adversarial review razona *sobre* el código, el QA engine lo **ejecuta**. Vive en `build-local.mjs` y corre vitest con coverage v8 sobre la lógica de dominio.

5.1 De dónde salen los tests — `genTests`

Los tests no nacen en build-local; nacen en **finalize** (`app-builder.ts` , `genTests`), durante la generación. `genTests` :

- Selecciona hasta 6 candidatos: `lib/*.ts` que **no** sean tests, que no estén en el skip-set (`db|auth|mailer|rate|rateLimit|analytics|i18n|rentals/` — infraestructura del spine y módulos que ya traen sus propios tests), y que **exporten al menos una función** (regex sobre `export function|export const x = ( )`).

- Pide a `MODELS.code` un archivo `lib/__tests__/domain.test.ts` que testee la lógica **pura** del producto, inyectando `skillFor("test")` (el playbook `TEST` destilado de addyosmani).
- Las hard rules del prompt: importar las funciones REALES desde su path `@/lib/...`, testear sólo funciones puras (nunca llamar nada que toque DB/red), asertar **valores exactos** + casos borde, codificar las reglas de negocio del producto, prohibido `expect(true).toBe(true)`, snapshots, returns hardcoded o debilitar un assert para pasar.

Los módulos deterministas verticales — el más claro es `rentals-module.ts`, parametrizado al nombre real de la tabla detectada, **sin un solo hardcoding** — **shippean sus propios tests** (anti-double-booking vía `EXCLUDE USING gist`, pricing en integer-cents, refund-by-policy-snapshot, reviews double-blind). Por eso `genTests` los saltea: ya vienen con suites verificadas. Así, el piso de testing no depende de que el modelo local invente buenos tests para las primitivas más críticas — esas son determinísticas y probadas.

5.2 La medición — `runVitest`

`runVitest()` borra el output viejo, corre `npm vitest run --coverage --reporter=json` (timeout 180 s), y parsea **dos** artefactos: `vitest-results.json` para `total/passed/failed`, y `coverage/coverage-summary.json` para `cov` (line-coverage % total) y `perFile` (cobertura por archivo). El `vitest.config.ts` que escribe `assemble()` apunta el coverage a `lib/**/*.ts`, excluyendo tests — es decir, mide la cobertura de la **lógica de dominio**, no del andamiaje.

6. Queen Evidence Review

La diferencia entre "corrí tests" y "el dominio está probado" es la **cobertura medida contra una barra**. Esa es la Queen evidence review: la Abeja Reina del equipo no acepta la palabra del swarm, **mide la evidencia** y rebota el build con feedback específico si no llega.

6.1 La barra y el loop

```
COV_BAR    = PUGLIT_COVERAGE_BAR    (default 70)
QA_ROUNDS  = PUGLIT_QA_ROUNDS        (default 2)
```

`runTests()` orquesta:

1. **Bootstrap.** Si no se entregó ningún test, "*la Reina exige cobertura del dominio*" y se genera una ronda inicial con `writeTestsFor(domainLibs())`. Si aun así no hay lógica de dominio testeable, registra coverage 0% y termina honestamente.
2. **Evidencia inicial.** `runVitest()` → log `QA: X/Y tests · COVERAGE Z% (bar 70%)`.
3. **Bounce loop** mientras `failed > 0` o `cov < COV_BAR`, hasta `QA_ROUNDS`: - `uncovered(perFile)` calcula los archivos de dominio bajo la barra. `domainLibs()` lista `lib/*.ts` excluyendo el `SKIP_LIB` (infra del spine + tests). Sólo cuentan los que están en ese set Y cuya cobertura de líneas medida es `< COV_BAR`. - La Reina rebota con feedback **concreto** — no "mejorá", sino los archivos y sus porcentajes: 🐝 `REVIEW Abeja Reina ronda N: "X/Y verdes, coverage Z% < bar 70% — mostrame evidencia, falta cubrir: lib/pricing.ts (41%), lib/refund.ts (0%)"`. Devuelto al equipo. - El equipo escribe más tests con `writeTestsFor(gaps)` y se **re-mide** (`runVitest`).

4. **Veredicto.** `approved = failed === 0 && cov >= COV_BAR` . Se loguea el veredicto (✓ APROBADO por la Reina / ✗ NO alcanza el bar (entregado con deuda de QA)) y se persiste en el status: `{ tests, testsPassed, testsFailed, coverage, coverageBar, qaRounds, qaApproved }` .

```
runVitest → cov 41% < 70% → 🏰 "faltan lib/pricing.ts (41%), lib/refund.ts (0%)"
      |
      |   re-medir ← writeTestsFor(gaps)   (hasta QA_ROUNDS rondas)
      |
      |
cov ≥ 70% && failed == 0 → ✓ APROBADO por la Reina
```

6.2 `writeTestsFor` — escribir donde falta

`writeTestsFor(rels)` toma hasta 6 archivos, y por cada uno pide a Ollama (modelo local, vía `ask`) un archivo de test vitest con la disciplina **Arrange-Act-Assert** embebida en el system prompt: nombrar por comportamiento (`describe("<unit>")` , `it("<expected> when <condition>")`), una conducta por test, importar las funciones reales del alias `@/...` , testear sólo funciones puras (saltar lo que toca DB/red), asertar valores exactos + casos borde. Sólo escribe el archivo si vuelve algo con `describe|it|expect` . Crucialmente, apunta a los archivos que la Reina marcó como descubiertos — la generación de tests está **dirigida por la evidencia**, no es a ciegas.

La asimetría es deliberada: `genTests` (finalize) usa `MODELS.code` y produce un único archivo inicial; `writeTestsFor` (build-local) usa el modelo local de `ask` y va **archivo por archivo contra los gaps medidos**. Es la diferencia entre "el equipo entregó tests" y "la Reina exige más donde la evidencia muestra que falta".

7. Testing Standards

Tanto `genTests` como `writeTestsFor` inyectan el mismo gen: el playbook **TEST** de `playbooks.ts` , distilado de addyosmani test-driven-development + testing-patterns. Su tesis es *"Tests are PROOF, not decoration."*

7.1 Lo que un test DEBE ser

- **Arrange → Act → Assert** estructurado, una conducta por test, nombrado por **comportamiento**.
- **Unit tests** (vitest) de las reglas puras: pricing, availability/overlap, refund-by-policy, la state machine, validadores. Asertar **valores exactos** (integer cents), límites, y los casos de error.
- **Business-rule tests** que codifican las invariantes del producto: *"una reserva confirmada solapada se rechaza"*, *"el total de búsqueda === el total de checkout"*, *"el refund usa el SNAPSHOT de la política, no la actual"*.
- **Integration tests** que manejan las rutas API reales (register → act → assert status + efecto en DB), incluyendo casos adversariales (double-booking, fechas inválidas, over-capacity, actor equivocado).
- Los tests importan **los mismos helpers de Lib/** que usa la app, y reportan el número de cobertura.

7.2 Los anti-patterns (de addyosmani) — nunca

El playbook los enumera como prohibiciones explícitas, y los prompts de generación las repiten:

Anti-pattern	Por qué es prohibido
Testear detalles de implementación	testear inputs/outputs, no internals
Snapshot-everything	asertar valores específicos, no blobs
Estado mutable compartido	setup/teardown por test
Asserts demasiado amplios	hay que ser específico para cazar regresiones
<code>test.skip</code> / borrar un assert para pasar	arreglar o borrar, nunca debilitar
<code>await</code> faltante en async	falsos passes
Testear código de terceros	no es tu dominio
<code>expect(true).toBe(true)</code>	placeholder vacío, prueba nada

La conexión con la Queen es directa: estos anti-patterns son la forma típica de **inflar cobertura sin probar nada**. La Reina los neutraliza midiendo cobertura *real* de líneas — un test vacío no cubre ramas, así que no mueve la aguja, y el bounce loop lo expone.

8. Integration in the Pipeline

Las dos capas corren en **momentos distintos** del ciclo de vida, y es importante no confundirlos.

Etap	Dónde	Qué corre	Modelo	Verdict
finalize (generación)	<code>app-builder.ts</code> , <code>buildAdvance</code>	<code>genTests</code> → <code>adversarialReview</code>	<code>MODELS.code</code> (tests) + <code>MODELS.premium</code> ×3 (lentes) + <code>MODELS.code</code> (repair)	SHIP / CAVEATS / BLOCK + métricas
build-local (runtime gate)	<code>build-local.mjs</code>	<code>runTests</code> (vitest+coverage) + Queen + smoke test	Ollama local vía <code>ask</code>	<code>qaApproved</code> + <code>smoke</code> en el status

8.1 En finalize

El orden dentro de `buildAdvance`, ya con los archivos generados y los módulos inyectados:

```
... harvestModules
→ genTests           (escribe lib/__tests__/domain.test.ts en los files)
→ reconcile/integrate page routes
→ runSwarmChecks     (security + consistency estáticos)
→ repairPhantomTables (si hay phantom-table) → re-scan
→ repairSecurityWithFrontier (si hay high-sev security) → re-scan
→ adversarialReview   ← 3 lentes + cross-exam + bounded repair + verdict
    └─ si repara algo → runSwarmChecks otra vez
→ recordMetric(build_success / adverse_verdict)
→ storeExemplar (si el gate estático está limpio)
```


El comentario en el código es preciso: el adversarial review es "*external eyes before it reaches the judge — and it never trusts a claim, it reads the actual code.*" Corre **después** de los scans estáticos y de las reparaciones determinísticas (phantom-table, security frontier), de modo que las tres lentes ven el deliverable ya saneado de lo obvio y se concentran en lo semántico: math de dominio, abuso, salud de diseño. Su verdict y sus métricas alimentan después al juez genético del tournament (sección 04).

8.2 En build-local

`build-local.mjs` es el equivalente local de "*entregar a GitHub + CI*". Su secuencia, después de ensamblar el proyecto:

```
loadAppSchemaRepairing  (SQL en una DB fresca, repara DDL)
→ repairTs              (tsc → fix por archivo → loop, anti-oscilación)
→ runTests              ← vitest + coverage + Queen evidence review (sección 6)
→ (BYO deploy si hay tokens)
→ next dev + smokeTest ← RUNTIME GATE: boot + GET a cada página estática, 0 5xx
```

Acá la Queen corre sobre tests que **realmente se ejecutan** contra una base Postgres efímera cargada con el schema reparado — es ejecución, no análisis. El `genTests` de finalize sembró el primer archivo; build-local lo corre, lo mide, y la Reina lo hace crecer hasta la barra antes de que el smoke test confirme que la app, además de estar probada, arranca y sirve.

8.3 La unidad de las dos capas

Las dos responden a la misma pregunta desde dos ángulos: **¿este software se sostiene si lo escruta un par de ojos externos?** El adversarial review lo escruta *razonando* (tres personas hostiles, cross-examinadas, sin confiar en la afirmación). El QA engine lo escruta *midiendo* (vitest + coverage real, con la Reina rebotando lo que no llega). Ninguna de las dos premia la confianza del generador; ambas exigen evidencia. Es la misma idea del runtime gate llevada de la ejecución a la corrección: *medir, no creer*.